

Java Threads Assignment

Simulate Concurrent UPI Payments Using Threads and Synchronization

Objective:

To simulate a real-world scenario where a merchant is receiving UPI payments from multiple customers at the same time and must ensure the **account balance is updated correctly using proper thread synchronization**.

Problem Statement:

Imagine you are building a backend system for a merchant's UPI payment gateway. Multiple customers are sending payments to the merchant simultaneously. You must ensure that each payment is **processed safely**, and the **merchant's account balance** reflects all payments accurately — **without any race conditions**.

Requirements:

1. Create a MerchantAccount class with:
 - A field balance (initially 0).
 - A method receivePayment(double amount, String customerName) that:
 - Adds the amount to the balance.
 - Prints the customer name, amount received, and updated balance.
2. Create a CustomerPayment class that extends Thread:
 - Has a reference to the MerchantAccount.
 - Accepts customerName and paymentAmount as input.
 - Calls the receivePayment() method in its run().
3. In the main() method:
 - Create a single shared MerchantAccount instance.
 - Create and start 10 CustomerPayment threads with different names and amounts (e.g., ₹100 to ₹1000).
 - Wait for all threads to complete using .join().
4. Use **proper synchronization** to ensure no race conditions during balance updates.

Expected Output Format (Sample):

```
Customer: Ramesh paid ₹500. Updated Balance: ₹500.0
Customer: Sita paid ₹200. Updated Balance: ₹700.0
Customer: John paid ₹300. Updated Balance: ₹1000.0
...
Final Balance: ₹<total sum of all payments>
```

Synchronization Goal:

Ensure that:

- No two threads update the balance simultaneously.
- No payment is lost or misrepresented in the final balance.